CS3212 Algorithms

Aya Zirikly
11/12/2026

Real world Applications

Dijkstra (shortest path from one source):

- GPS navigation: the fastest route from your location to a destination, with road lengths or travel times as weights.
- Flight search: the cheapest connection from one airport to others, with fares or times as weights.

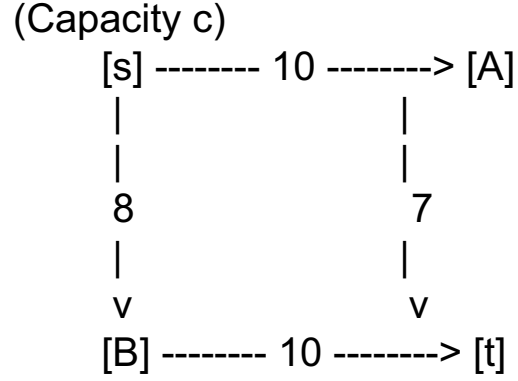
Prim (minimum spanning tree, connect everything cheaply):

- Laying cable or pipes: connect all the towns or buildings with the least total cable, starting from one site and growing the network.
- Power grid or fiber network planning: connect all substations or offices with the least total wire.

Network Flow (Edmonds-Karp) & Dynamic Programming Foundations

Introduction to Flow Networks

- A **Flow Network** is a directed graph G = (V, E) where:
- Every edge (u, v) has a non-negative **Capacity** c(u, v) >= 0. If (u, v) is not an edge, c(u, v) = 0.
- Two special vertices are designated: a **Source s** (in-degree 0, generates flow) and a **Sink t** (out-degree 0, consumes flow).



The Two Fundamental Rules of Flow

For a flow function f(u, v) assigning a real number to each edge:

1. Capacity Constraint

For every edge (u, v), the flow cannot exceed capacity:

0 <= f(u, v) <= c(u, v)

2. Flow Conservation Constraint

For every vertex u except source s and sink t, total incoming flow must equal total outgoing flow:

Sum of f(v, u) for all incoming v = Sum of f(u, w) for all outgoing w

The Two Fundamental Rules of Flow

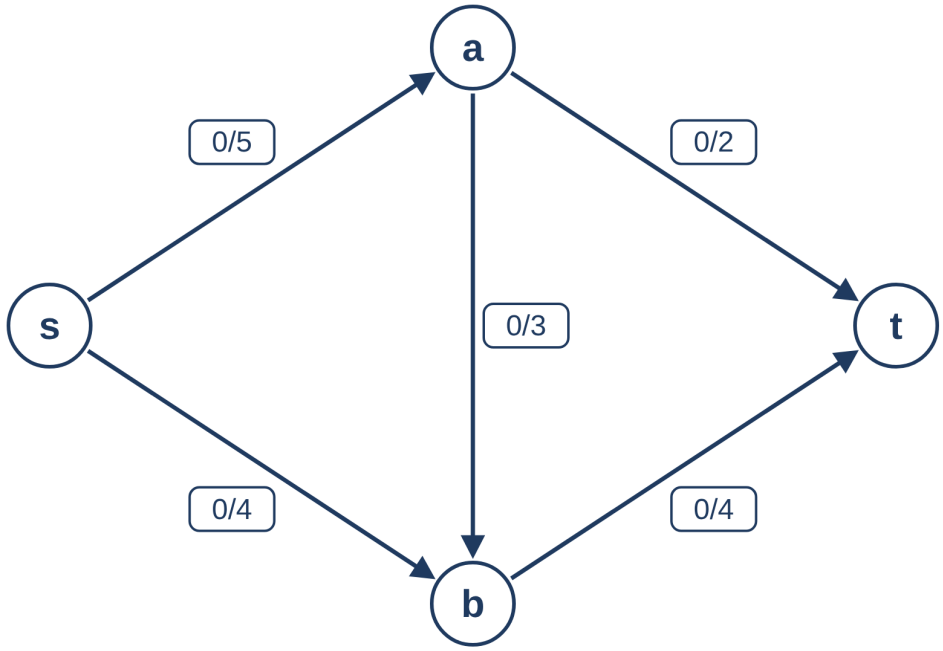
Total Network Flow Value |f|

The total flow leaving the source s, which equals the total flow arriving at sink t:

|f| = Sum of f(s, v) for all outgoing edges from s

Example flow network G

Source s, sink t; each edge labelled flow / capacity (start with f = 0)



Flow rules

Capacity: $0 \le f(u,v) \le c(u,v)$

Conservation: flow in = flow out
at every vertex except s and t

Value $|f|$ = total flow leaving s

Goal: maximise $|f|$

Ford–Fulkerson: while G_f has an s → t path, push its bottleneck amount along it.

Capacities here: s→a 5, s→b 4, a→b 3, a→t 2, b→t 4

Residual Networks & Augmenting Paths

Residual Capacity Formula

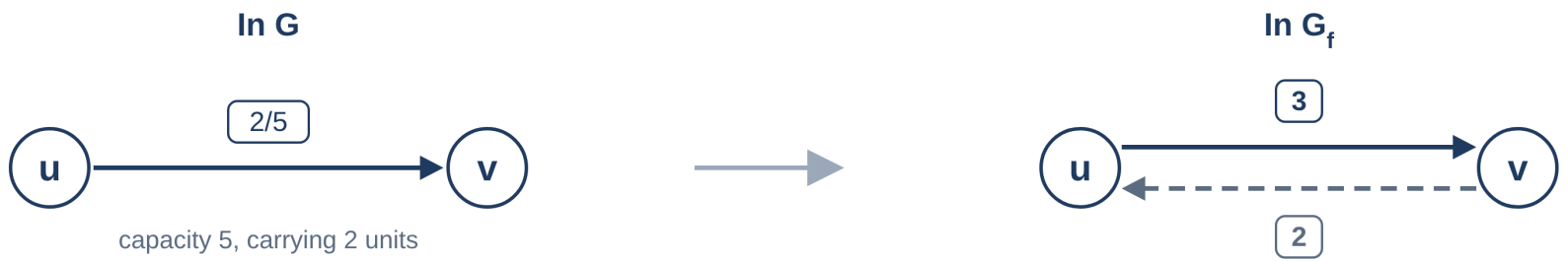
Given a flow network G and a current flow f, the **Residual Network G_f** consists of the same vertices but uses **Residual Capacities r(u, v)**:

$r(u, v) = c(u, v) - f(u, v)$ [Forward Edge: Remaining unused capacity]

$r(v, u) = f(u, v)$ [Backward Edge: Ability to undo/cancel flow]

From G to the residual graph G_f

Each edge with flow f and capacity c gives up to two residual edges



Forward residual edge u → v

residual capacity c(u,v) − f(u,v) = 5 − 2 = 3

“we can still push 3 more units u → v”

Present only if c − f > 0

——— drawn solid

Backward residual edge v → u

residual capacity f(u,v) = 2

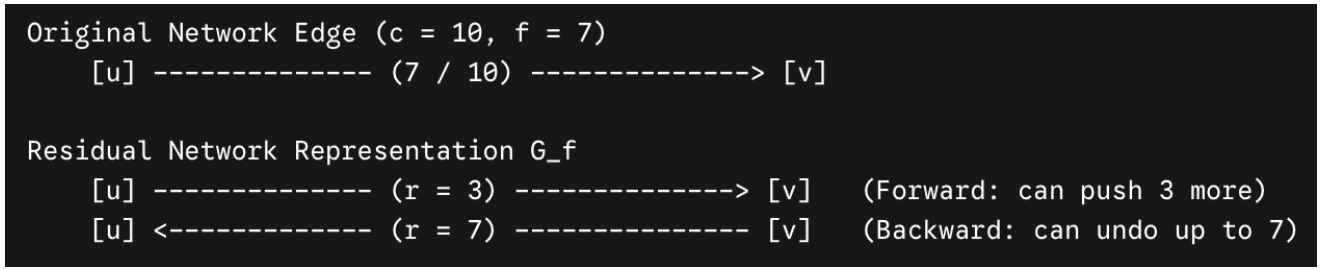
“we can cancel (undo) up to 2 units of u → v”

Present only if f > 0

- - - drawn dashed

Why Backward Edges Are Essential

- Backward edges allow the algorithm to **undo bad prior greedy choices**.



- **Augmenting Path:** A simple directed path from source s to sink t in the residual graph G_f where every edge has residual capacity r(u, v) > 0.
- **Bottleneck Capacity:** The minimum residual capacity along an augmenting path P:

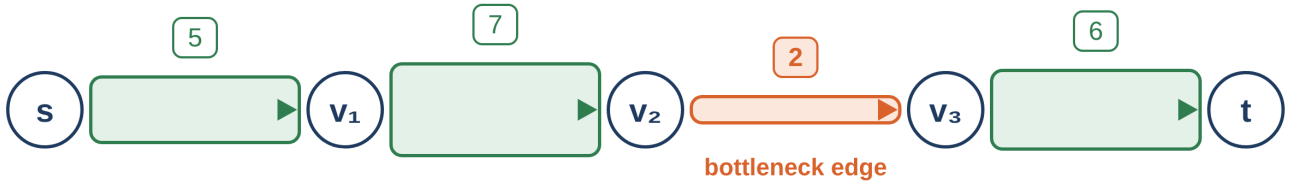
$$b = \min \{ r(u, v) \text{ for all edges } (u, v) \text{ in } P \}$$

Bottleneck capacity: before and after augmenting

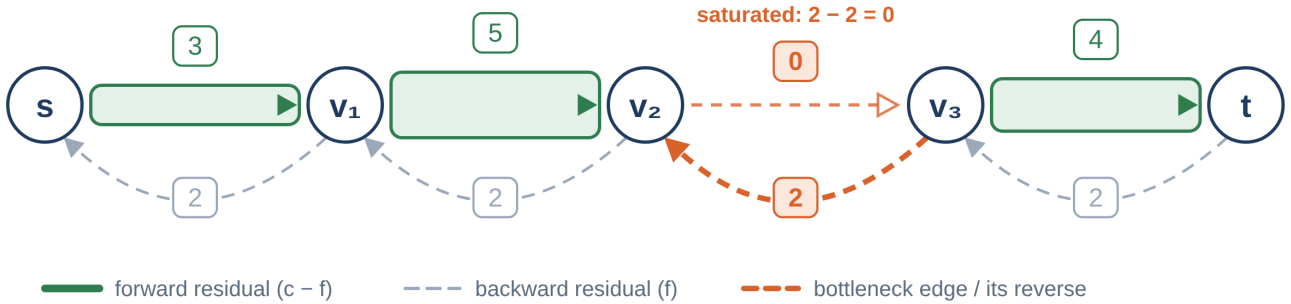
Residual capacities in G_f along $P = s \to v_1 \to v_2 \to v_3 \to t$, before and after pushing $\Delta = 2$

Before

Δ not yet pushed



After pushing
Δ = 2



Δ = bottleneck(P) = min(5, 7, 2, 6) = 2

After pushing $\Delta = 2$, the bottleneck's forward residual $v_2 \to v_3$ becomes $2 - 2 = 0$ (it leaves G_f), and the backward arc $v_3 \to v_2$ gets residual capacity 2. Every other edge on P loses 2 forward and gains a backward arc of 2.

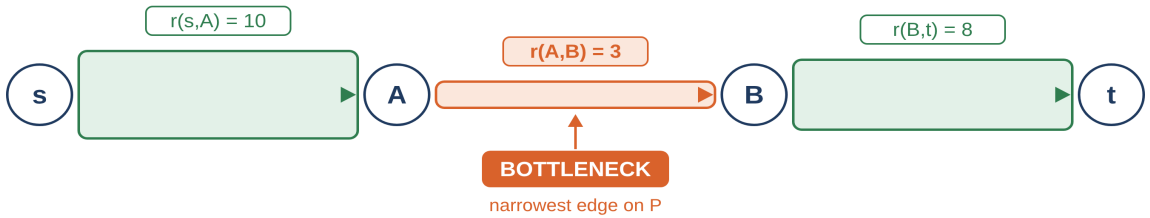
Bottleneck Capacity Example

1. Augmenting Path P in Residual Network G_f

- Suppose BFS discovers the path P: s -> A -> B -> t with the following remaining residual capacities:

Bottleneck of an augmenting path

Augmenting path P = s → A → B → t in the residual graph G_f; pipe width ∝ residual capacity r



Δ = bottleneck(P) = min(r(s,A), r(A,B), r(B,t)) = min(10, 3, 8) = 3

The path can carry only as much as its narrowest residual edge, so we push Δ = 3 units along every edge of P.

Afterwards A → B has residual capacity 3 − 3 = 0 (saturated), while s → A and B → t keep 7 and 5.

Bottleneck Capacity Example

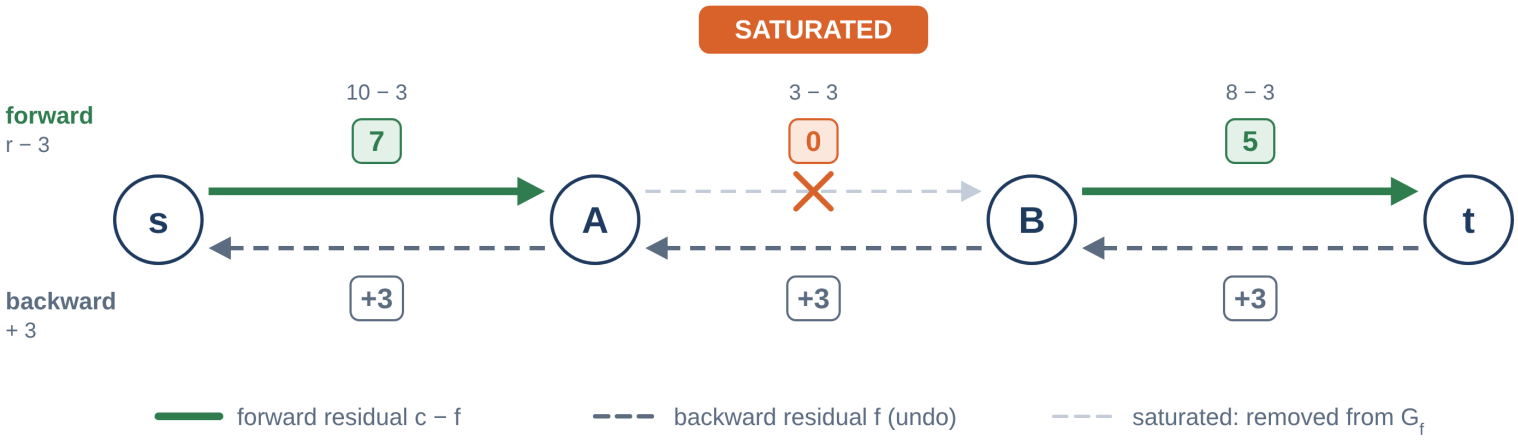
2. Computing Bottleneck Capacity b

- The maximum additional flow we can push along the entire path P is limited by its narrowest edge:
- b = min { r(s, A), r(A, B), r(B, t) }
- b = min { 10, 3, 8 } = 3

We cannot push more than 3 units along P, or edge (A, B) would exceed its capacity.

After pushing Δ = 3 along P

Residual capacities on s → A → B → t in the new G_f



Forward: r(u,v) ← r(u,v) − 3 **Backward: r(v,u) ← r(v,u) + 3**

s → A: 10 − 3 = 7, A → B: 3 − 3 = 0 (saturated, drops out of G_f), B → t: 8 − 3 = 5.

Each reverse edge A → s, B → A, t → B gains 3 units of capacity, so a later path can cancel this flow.

Bottleneck Capacity Example

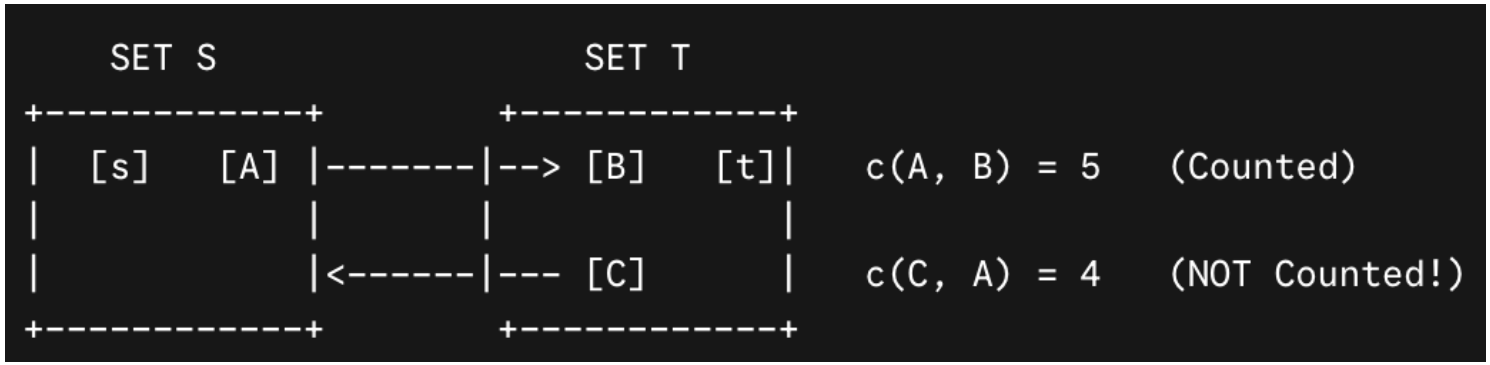
3. Updating Residual Capacities After Pushing b = 3

- For every edge (u, v) on path P, subtract b from the forward residual capacity and add b to the backward residual capacity:
  - Edge (A, B) now has 0 forward residual capacity (it is **saturated** and cannot carry more forward flow until some flow is undone).
  - Total network flow |f| increases by +3.

The Max-Flow Min-Cut Theorem

Definitions

- **s-t Cut (S, T):** A partition of vertices V into two disjoint sets S and T such that source s is in S and sink t is in T.
- **Capacity of Cut C(S, T):** The sum of capacities of edges pointing **from S to T**:
- **c(S, T) = Sum of c(u, v) for all u in S AND v in T**

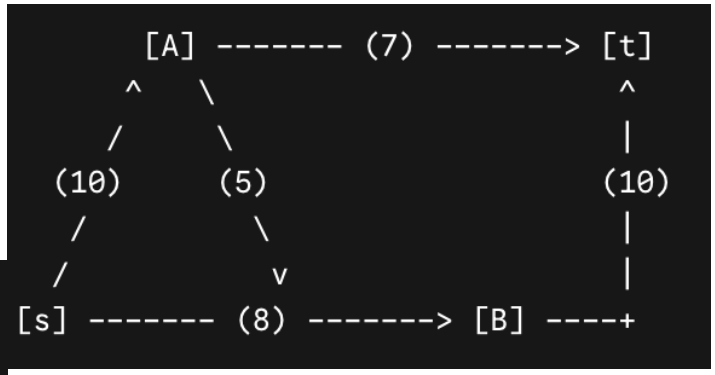


Cuts, Minimum Cut & The Flow Upper Bound

Competing Optimization Goals

- **The Maximum Flow Problem:** Find a valid flow f that maximizes total flow |f| leaving source s.
- **The Minimum Cut Problem:** An **s-t Cut (S, T)** partitions vertices V into two sets where s is in S and t is in T.
  - **Cut Capacity:** c(S, T) = Sum of c(u, v) for all edges pointing **from S to T** (ignore T -> S).
  - **Minimum Cut:** The s-t cut (S, T) with the **smallest capacity** among all possible cuts.

Example



Cut Partition (S | T)	Forward Edges (S -> T)	Cut Capacity c(S, T)
S = {s} | T = {A, B, t}	(s, A):10, (s, B):8	10 + 8 = 18
S = {s, A} | T = {B, t}	(s, B):8, (A, B):5, (A, t):7	8 + 5 + 7 = 20
S = {s, B} | T = {A, t}	(s, A):10, (B, t):10	10 + 10 = 20
S = {s, A, B} | T = {t}	(A, t):7, (B, t):10	7 + 10 = 17 <-- Min-Cut!

The Ford-Fulkerson Method

Why Is It Called a "Method" Instead of an "Algorithm"?

- **Ford-Fulkerson** is a general algorithmic framework because it leaves **how to find an augmenting path** unspecified (you could use DFS, BFS, random search, or maximum-bottleneck search).

```
Algorithm Ford-Fulkerson(G, s, t):
    1. Initialize flow f(u, v) = 0 for every edge (u, v) in E
    2. Build the initial residual network G_f

    3. while there exists an augmenting path P from s to t in G_f:
            // Find the bottleneck capacity along path P
            b = min { r(u, v) for all edges (u, v) in P }

            // Augment flow along path P
            for each edge (u, v) in P:
                if (u, v) is a forward edge in original graph G:
                    f(u, v) = f(u, v) + b        // Push more flow forward
                else:
                    f(v, u) = f(v, u) - b        // Cancel/reroute prior flow

            Update residual capacities in G_f

    4. return f
```

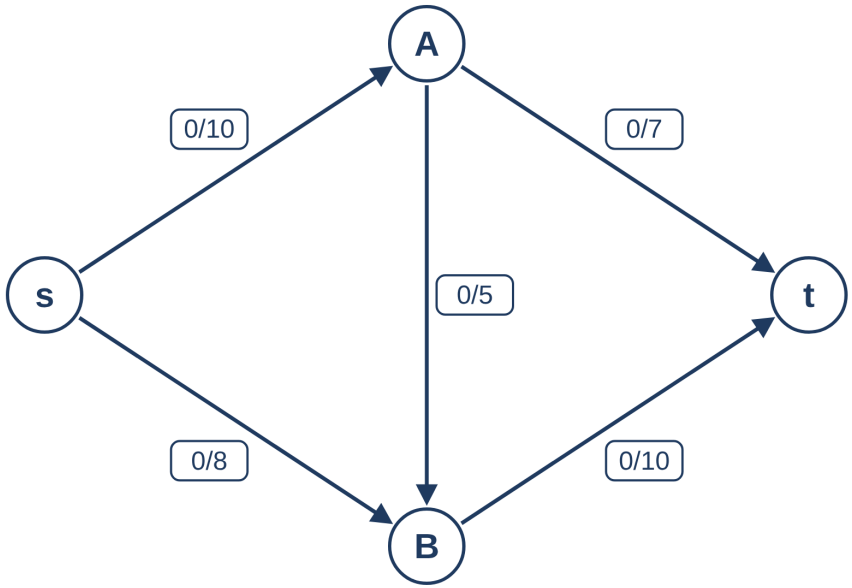
The Edmonds-Karp Algorithm & Full Execution Trace

Edmonds-Karp is a specific implementation of the general Ford-Fulkerson method:

- **Key Choice:** Always select the augmenting path in G_f using **Breadth-First Search (BFS)** (i.e., the path with the **fewest number of edges**).

Edmonds–Karp on flow network G

Source s, sink t; each edge labelled flow / capacity (start with f = 0)



Edmonds–Karp

Ford–Fulkerson, but each augmenting path is found by BFS in G_f:

the s → t path with the fewest edges.

repeat: build G_f
BFS from s to find path P
push Δ = bottleneck(P)
until t is unreachable

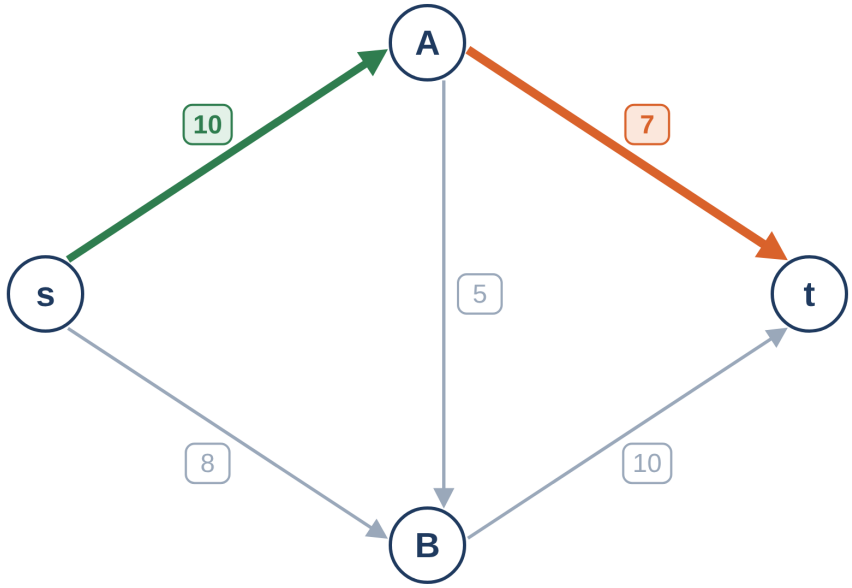
Running time $O(V \cdot E^2)$

Capacities: s→A 10, s→B 8, A→B 5, A→t 7, B→t 10

BFS scans neighbours in the order s, A, B, t.

Iteration 1: BFS finds s → A → t

Residual graph G_f for the current flow (|f| = 0); edges with residual capacity 0 are not drawn



BFS from s

L0 {s} · L1 {A, B} · L2 {t}

Shortest path: 2 edges

Residual capacities on the path

s → A	forward	10	
A → t	**forward**	**7**	**← bottleneck**

Δ = min(10, 7) = 7

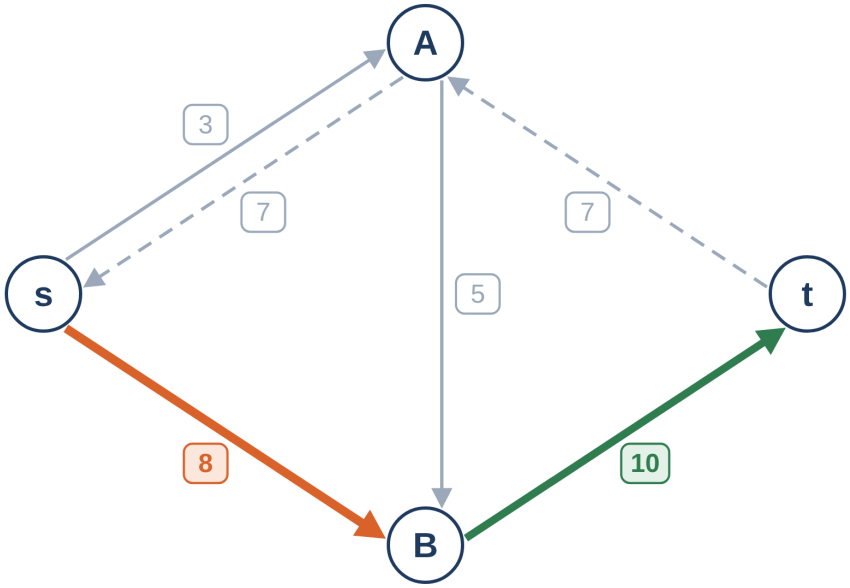
Push 7 units along the path.

|f|: 0 → 7

- BFS augmenting path edge
- bottleneck edge (min residual cap.)
- forward residual edge (c − f)
- backward residual edge (f)

Iteration 2: BFS finds s → B → t

Residual graph G_f for the current flow (|f| = 7); edges with residual capacity 0 are not drawn



BFS from s

L0 {s} · L1 {A, B} · L2 {t}

Shortest path: 2 edges

Residual capacities on the path

s → B	**forward**	**8**	**← bottleneck**
B → t	forward	10	

Δ = min(8, 10) = 8

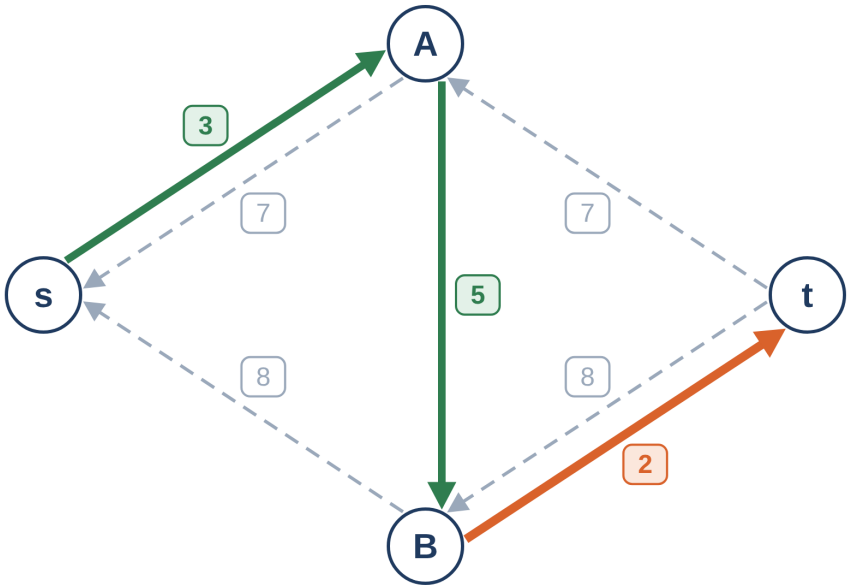
Push 8 units along the path.

|f|: 7 → 15

- BFS augmenting path edge
- bottleneck edge (min residual cap.)
- forward residual edge (c − f)
- backward residual edge (f)

Iteration 3: BFS finds s → A → B → t

Residual graph G_f for the current flow (|f| = 15); edges with residual capacity 0 are not drawn



BFS from s

L0 {s} · L1 {A} · L2 {B} · L3 {t}

Shortest path: 3 edges

Residual capacities on the path

s → A	forward	3	
A → B	forward	5	
B → t	**forward**	**2**	**← bottleneck**

Δ = min(3, 5, 2) = 2

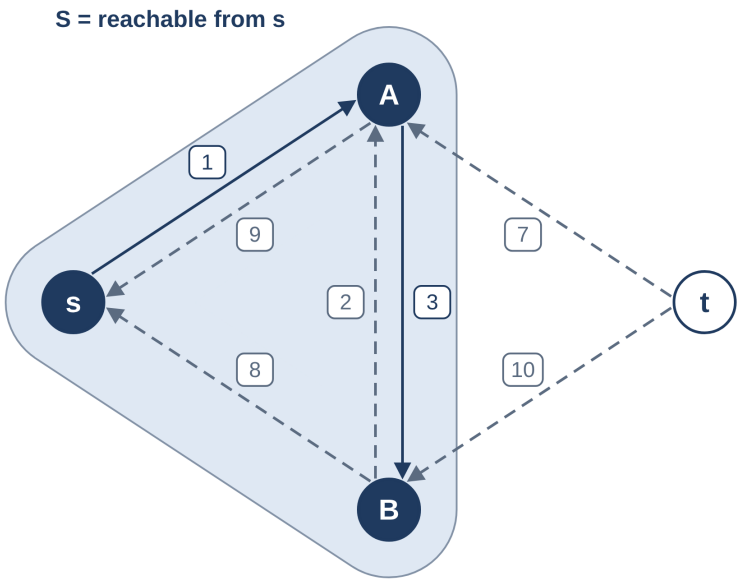
Push 2 units along the path.

|f|: 15 → 17

- BFS augmenting path edge
- bottleneck edge (min residual cap.)
- forward residual edge (c − f)
- backward residual edge (f)

BFS cannot reach t: the flow is maximum

Final residual graph G_f: BFS from s reaches only {s, A, B}



Final BFS from s

L0 {s} · L1 {A} · L2 {B}

Edges leaving S in G_f:

- A → t: residual 7 − 7 = 0 (gone)
- B → t: residual 10 − 10 = 0 (gone)

Only backward edges t → A, t → B remain, and they point into S.

t is unreachable: Edmonds–Karp stops.

Final |f| = 17

S = vertices reachable from s in G_f

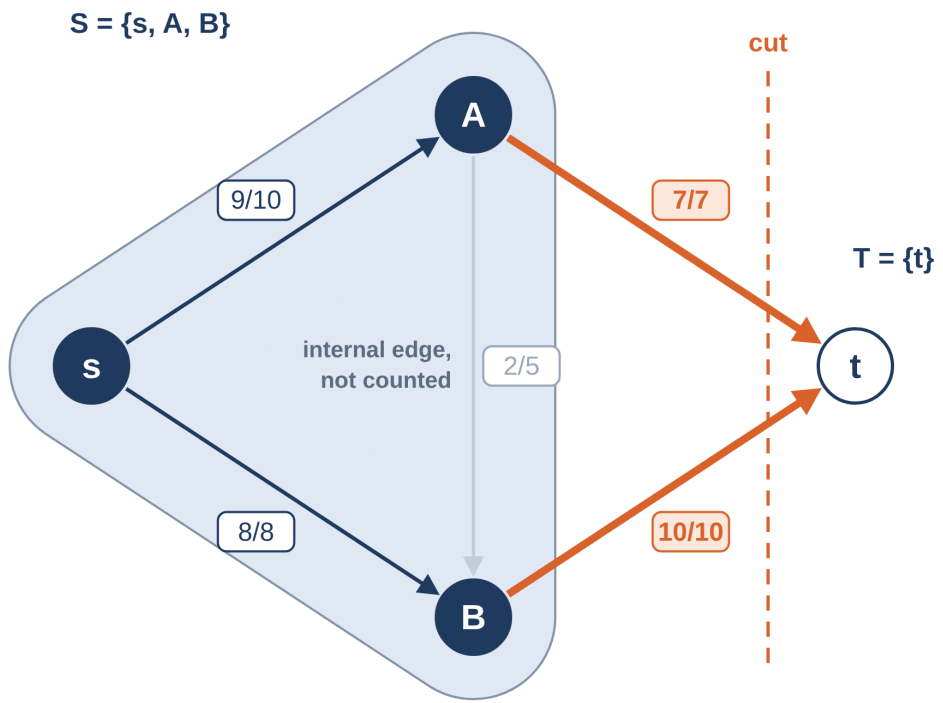
forward residual edge (c − f)

backward residual edge (f)

- A→t and B→t are both full, so we can’t get to t.
- The flow can’t grow. The full edges leaving the shaded area add up to 7 + 10 = 17, which is the flow we already sent.”

Max flow = min cut = 17

Only edges crossing from S to T count toward the cut; A → B stays inside S



Min cut (S, T)

Cut edges S → T (all saturated):

- **A → t: 7/7**
- **B → t: 10/10**

c(S, T) = 7 + 10 = 17

Flow out of s (s → A, s → B):

|f| = 9 + 8 = 17

|f| = 17 = c(S, T)

A → B (2/5) has both ends in S, so it is not part of the cut.

No flow exceeds any cut capacity, so this flow is maximum and this cut is minimum.

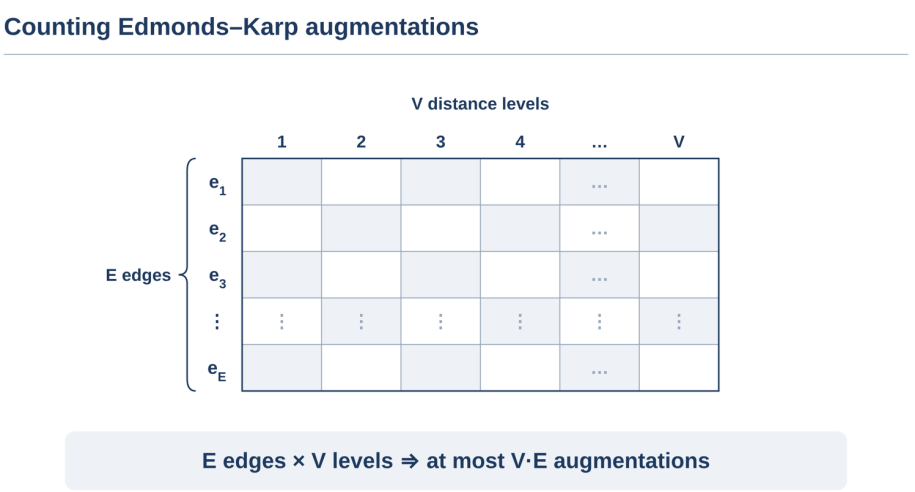
- source side S of the cut
- cut edge S → T (counted)
- cut boundary
- internal edge inside S (not counted)

Complexity Analysis

Edmonds–Karp runs in $O(V \cdot E^2)$ time. V is the number of vertices and E is the number of edges.

Where it comes from:

- **One search costs O(E).** Each round uses BFS, which looks at each edge about once.
- **There are at most about V·E rounds**



Complexity Analysis

Edmonds–Karp runs in $O(V \cdot E^2)$ time. V is the number of vertices and E is the number of edges.

Where it comes from:

- **One search costs O(E).** Each round uses BFS, which looks at each edge about once.
- **There are at most about V·E rounds.** Each round fills at least one edge. The shortest paths only get longer over time, so each edge can fill only a limited number of times.
- **Multiply.** Rounds × cost per round = $(V \cdot E) \times E = V \cdot E^2$ -- It doesn't depend on the capacities.

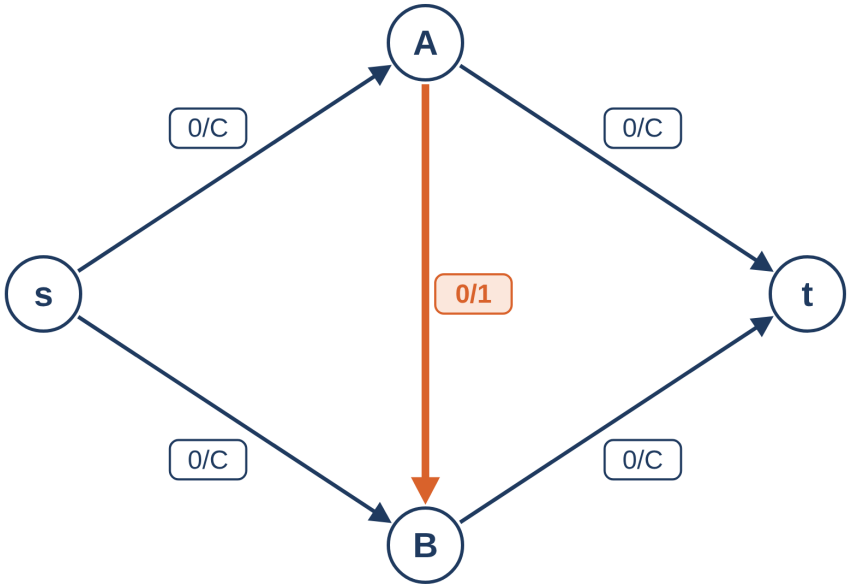
Complexity Analysis: Why BFS Matters

Ford-Fulkerson vs. Edmonds-Karp

Feature	Edmonds-Karp (BFS)
Path Selection	Shortest Path in Edges (via BFS)
Time Complexity	O(V * E^2)
Dependency	INDEPENDENT of Capacity Values
Path Length Behavior	Distance to u in G_f never decreases

When does the choice of augmenting path matter?

Same shape as before, but four huge capacities C and one tiny middle edge A → B of capacity 1



The setup

C = 1,000,000 on s→A, s→B, A→t, B→t

capacity 1 on A → B

Max flow = 2C = 2,000,000

(send C along s→A→t and C along s→B→t)

Generic Ford–Fulkerson lets any s → t path in G_f be the augmenting path.

An unlucky (e.g. DFS) choice keeps using **the middle edge, so each round pushes only 1.**

Running time O(E · |f*|) depends on C!

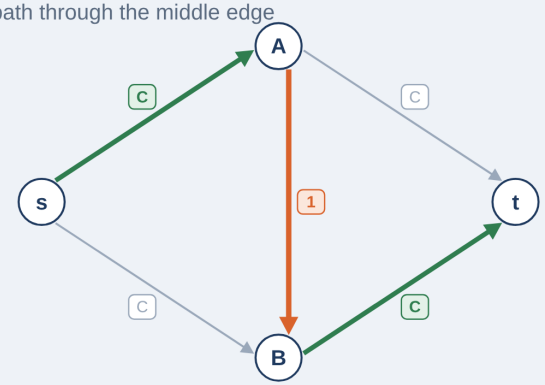
the 1-capacity middle edge A → B

Generic Ford–Fulkerson: the zigzag (DFS picks badly)

Each path uses the middle edge, so its bottleneck is 1 and only 1 unit moves per round

Round 1: s → A → B → t (3 edges)

forward path through the middle edge

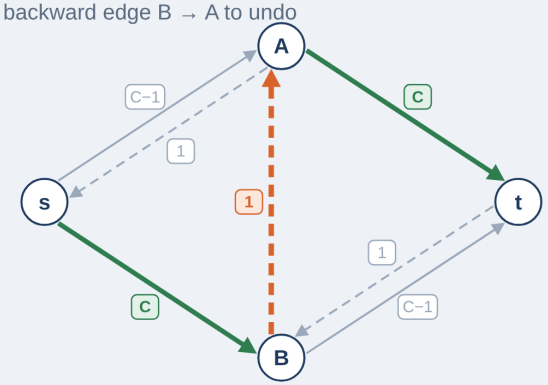


1. Residual: s→A = C, A→B = 1, B→t = C
2. Bottleneck = min(C, 1, C) = 1 on A→B
3. **Push 1: flow s→A = 1, A→B = 1, B→t = 1**

|f| goes 0 → 1

Round 2: s → B → A → t (3 edges)

uses the backward edge B → A to undo



1. Residual: s→B = C, B→A = 1 (undo of A→B), A→t = C
2. Bottleneck = min(C, 1, C) = 1 on B→A
3. **Push 1: s→B = 1, A→B back to 0 (undone), A→t = 1**

|f| goes 1 → 2

After 2 rounds: |f| = 2, A → B is back to 0, and each outer edge carries 1 unit.

Round 3 has the same shape as round 1, so the pattern repeats, adding 1 unit per round.

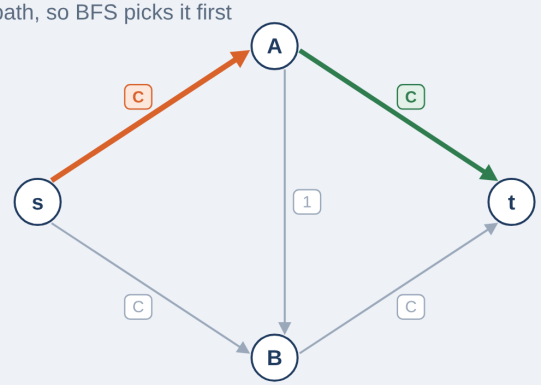
Total: 2C = 2,000,000 rounds to reach |f| = 2,000,000

Edmonds–Karp: BFS finishes in just 2 augmentations

BFS always returns a shortest s → t path in G_f, so the 3-edge paths through A → B are never chosen here

Round 1: BFS finds s → A → t (2 edges)

shortest path, so BFS picks it first

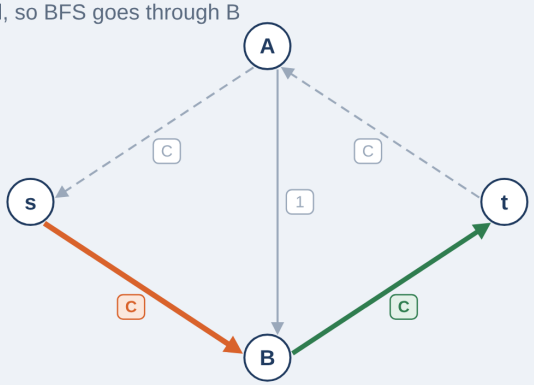


1. BFS from s: L1 {A, B}, L2 {t}; first path to t is s→A→t
2. Residual: s→A = C, A→t = C; bottleneck = C
3. **Push C: s→A = C, A→t = C**

|f| goes 0 → C

Round 2: BFS finds s → B → t (2 edges)

s→A is full, so BFS goes through B



1. BFS from s: only s→B has residual (s→A is full)
2. Then B→t: L1 {B}, L2 {t}; bottleneck = min(C, C) = C
3. **Push C: s→B = C, B→t = C**

|f| goes C → 2C

After round 2, s has no residual edges left, so BFS reaches only {s} and the flow is maximum.

|f| = 2C = 2,000,000 = c({s}, {A, B, t}) = C + C; the middle edge A → B is never used.

2 augmentations instead of 2,000,000

Complexity Analysis: Why BFS Matters

Ford-Fulkerson vs. Edmonds-Karp

Feature	General Ford-Fulkerson	Edmonds-Karp (BFS)
Path Selection	Any DFS / Arbitrary Path	Shortest Path in Edges (via BFS)
Time Complexity	O(E * |f*|)	O(V * E^2)
Dependency	Depends on Max Flow Value	INDEPENDENT of Capacity Values
Path Length Behavior	Can bounce back and forth	Distance to u in G_f never decreases

Symbols

- **f**: A valid flow function assigning a flow amount to every edge (u, v).
- **|f|**: The **value** of flow f—the total amount of flow leaving source s and arriving at sink t (here, the vertical bars denote *flow value*, not absolute value).
- **f***: The **optimal (maximum) flow** assignment (in algorithms notation, a * superscript or suffix denotes an optimal solution).
- **|f*|**: The **maximum flow value**—a single number representing the highest possible throughput of the network.

Introduction to Dynamic Programming

What is Dynamic Programming (DP)?

- **Dynamic Programming** is an algorithm design methodology that solves complex problems by **breaking them down into overlapping subproblems**, solving each subproblem **exactly once**, and **storing their solutions** in a lookup table.
- **Origin:** Termed by Richard Bellman in the 1950s. "Programming" here refers to *tabular planning*, not writing computer code.

Comparing Core Algorithmic Paradigms

Paradigm	Subproblem Structure	Key Strategy
Divide & Conquer	Independent, non-overlapping (e.g., Merge Sort, QuickSort)	Solve subproblems recursively, then combine the results
Greedy	One choice leaves one smaller problem (e.g., Prim, Kruskal, Dijkstra)	Make the locally best choice, without backtracking
Dynamic Programming	Dependent, highly overlapping (e.g., Floyd–Warshall, Knapsack)	Solve each subproblem once, store results for later lookup